DIGITAL CHEMISTRY

Michaelmas Term 2026

Dr Filip Szczypiński

filip.t.szczypinski@durham.ac.uk

Contents

1 Translating molecules for computers **7**

1.1 Information in chemical structures 7

1.2 Coordinates and molecular geometry 8

1.3 Molecules as graphs 11

1.4 Line notations 12

1.5 Identifiers and chemical records 14

1.6 Bit vectors and molecular fingerprints 17

1.7 Summary 20

2 Exploring chemical space **21**

2.1 Introduction 21

2.2 What is chemical space? 21

2.3 Molecular similarity and distance 21

2.4 Structural and property-based spaces 21

2.5 High-dimensional data 22

2.6 Principal component analysis 22

2.7 Other dimensionality-reduction methods 22

2.8 Clustering chemical data 22

2.9 Green chemistry case study: solvent space 23

2.10 Designing informative datasets 23

2.11 Chapter summary and practice 23

2.12 Summary 23

2.13 Questions for review 24

3 Visual methods for mechanistic investigations **25**

3.1 Introduction 25

3.2 Seeing mechanism in experimental data . . . 25
3.3 Linear free-energy relationships . . . 25
3.4 Asymmetric catalysis and non-linear effects . . . 25
3.5 Proline organocatalysis as chemical context . . . 26
3.6 Kagan, Soai and asymmetric amplification . . . 26
3.7 Reaction progress kinetics . . . 26
3.8 Reaction progress kinetic analysis . . . 26
3.9 Variable time normalisation analysis . . . 27
3.10 Mechanistic case study . . . 27
3.11 What has the evidence established? . . . 27
3.12 Chapter summary and practice . . . 27
3.13 Summary . . . 28
3.14 Questions for review . . . 28

4 Machine learning for chemistry: a primer 29

4.1 Introduction . . . 29
4.2 From physical organic models to machine learning . . . 29
4.3 Types of learning problem . . . 29
4.4 Chemical datasets . . . 29
4.5 Training, validation and testing . . . 30
4.6 Linear regression . . . 30
4.7 Learning non-linear relationships . . . 30
4.8 Evaluating models . . . 30
4.9 Overfitting and underfitting . . . 31
4.10 Applicability domain and uncertainty . . . 31
4.11 Green chemistry case study: solvent effects . . . 31
4.12 Chapter summary and practice . . . 31
4.13 Summary . . . 32
4.14 Questions for review . . . 32

5 Modern molecular descriptors 33

5.1 Introduction . . . 33
5.2 From representation to descriptor . . . 33

5.3 Classical physical-organic descriptors . . . 33
5.4 Steric descriptors . . . 33
5.5 Conformational dependence . . . 34
5.6 Electronic descriptors . . . 34
5.7 Descriptor generation at scale . . . 34
5.8 Fingerprints versus physicochemical descriptors . . . 35
5.9 Learned representations . . . 35
5.10 Choosing descriptors for a task . . . 35
5.11 Chapter summary and practice . . . 35
5.12 Summary . . . 36
5.13 Questions for review . . . 36

6 Reaction optimisation: OVAT, DoE and Bayesian optimisation 37

6.1 Introduction . . . 37
6.2 Defining an optimisation problem . . . 37
6.3 One variable at a time . . . 37
6.4 Factorial experiments . . . 37
6.5 Response surfaces and design of experiments . . . 38
6.6 Sequential experimentation . . . 38
6.7 Bayesian optimisation . . . 38
6.8 Green multi-objective optimisation . . . 38
6.9 Failed and infeasible experiments . . . 39
6.10 Worked reaction-optimisation case study . . . 39
6.11 Chapter summary and practice . . . 39
6.12 Summary . . . 39
6.13 Questions for review . . . 40

7 Generating chemical data: high-throughput and flow chemistry 41

7.1 Introduction . . . 41
7.2 Why generate data at scale? . . . 41
7.3 Parallel experimentation . . . 41
7.4 High-throughput reaction screening . . . 41
7.5 The analytical bottleneck . . . 42

7.6 Flow chemistry fundamentals 42
7.7 Serial experimentation in flow 42
7.8 Batch versus flow automation 42
7.9 Data infrastructure 43
7.10 Does automation make chemistry greener? 43
7.11 Chapter summary and practice 43
7.12 Summary 43
7.13 Questions for review 44

8 Autonomous and self-driving laboratories 45

8.1 Introduction 45
8.2 Automation, autonomy and closed loops 45
8.3 The experimental closed loop 45
8.4 Hardware layer 46
8.5 Software and data layer 46
8.6 Decision layer 46
8.7 Closing the loop 46
8.8 Robustness and error handling 47
8.9 Human-in-the-loop chemistry 47
8.10 Green autonomous chemistry 47
8.11 What should laboratories automate? 47
8.12 Course synthesis and outlook 48
8.13 Summary 48
8.14 Questions for review 48

9 Workshop: molecules as data with RDKit 49

9.1 Introduction 49
9.2 Workshop aims and setup 49
9.3 Drawing and inspecting molecules 49
9.4 Substructure searching 49
9.5 Fingerprints and similarity 50
9.6 Calculating simple descriptors 50
9.7 Selecting a diverse mini-library 50

9.8 Workshop reflection 50
9.9 Summary 51
9.10 Questions for review 51

10 Revision class and example examination questions 52

10.1 Introduction 52
10.2 How the course fits together 52
10.3 How to approach examination questions 52
10.4 Example question: representation and chemical space 53
10.5 Example question: mechanism from data 53
10.6 Example question: modelling and validation 53
10.7 Example question: optimisation and automation 53
10.8 Final checklist 54
10.9 Summary 54
10.10 Questions for review 54

1. Translating molecules for computers

1.1. Information in chemical structures

Chemists routinely communicate molecular structure using drawings. A skeletal formula can encode a considerable amount of information in a compact form because we have learnt conventions for interpreting atoms, bonds, functional groups and stereochemistry. Consider a simple example of proline (Fig. 1.1) and what information we implicitly capture about its structure.

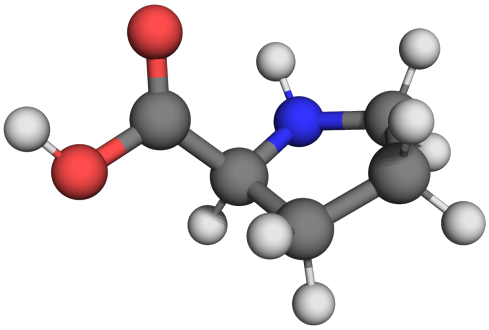


Figure 1.1. Molecular structure of proline.

- atoms (incl. implicit hydrogens)
- connectivity and bond orders
- functional groups
- stereochemistry
- protonation
- ...

Computers do not interpret chemical drawings in the same way. To store, search, compare and analyse molecules computationally, chemical information must first be represented in a form that can be processed by computer algorithms.

Main question: how should we represent a molecule to a computer?

Throughout this chapter, we will encounter several ways of representing molecules digitally. These range from three-dimensional atomic coordinates to text

strings, identifiers and numerical fingerprints. Rather than asking which representation is 'best', we will ask **what chemical information each representation contains**, what it discards, and what it allows us to do.

The information required by a computer depends on the task. A calculation of molecular geometry requires the positions of atoms in three-dimensional space. A substructure search requires information about atoms and their connectivity. Searching a database for a particular chemical requires a consistent identifier. Comparing the structural similarity of thousands of compounds requires a representation that can be compared efficiently. Consequently, the same molecule may be represented in several different ways within a single workflow.

For each task below, consider what information must be retained:

- calculate the distance between two atoms;
- determine whether a molecule contains a substructure;
- find a compound in a database;
- compare a molecule with one million other structures.

1.2. Coordinates and molecular geometry

One of the most direct ways of describing a geometry is to specify the position of every atom in three-dimensional space, often in a simple `.xyz` file (Lst. 1.1). The first line contains the number of atoms, followed by molecule name, and its coordinates. Other filetypes (*e.g.*, `.mol2`) might also include bonding information.

```
17
Proline
   H       -0.13097       -1.07655        1.63572
   C       -0.15726       -0.63902        0.60818
   C       -1.20401       -1.38526       -0.20437
   O       -1.15189       -1.78305       -1.35456
   O       -2.35852       -1.64177        0.45250
   H       -2.96245       -2.10328       -0.12133
   N        1.22788       -0.77675        0.06680
   C        1.69088        0.54717       -0.43489
   H        2.56627        0.82982        0.18087
   H        2.04876        0.49358       -1.48090
   C        0.54048        1.53722       -0.27264
   H        0.08541        1.78163       -1.25139
   H        0.88732        2.50157        0.14048
   C       -0.47304        0.86447        0.64762
   H       -1.51350        1.08299        0.34489
   H       -0.38097        1.24444        1.68280
   H        1.28559       -1.47722       -0.63981
```

Listing 1.1. Proline geometry as an `.xyz` file.

A Cartesian representation assigns three coordinates to each atom. Molecular geometries can alternatively be expressed using **internal coordinates**, such as bond

lengths, bond angles, and dihedral angles, in a *Z-matrix*. Quantum-chemical calculations and molecular simulations routinely use representations of this type. However, even such explicit representation might miss important details required for calculations, *e.g.*, overall charge and spin multiplicity (Lst. 1.2).

```
! B3LYP D3BJ def2-TZVP OPT FREQ

* xyzfile 0 1 proline.xyz
```

Listing 1.2. An ORCA input file for a geometry optimisation calculation of proline.

Now, consider the two structures of proline in Figure 1.2 – are they the same molecule? A useful molecular representation should not change just because we move or rotate a molecule in space.

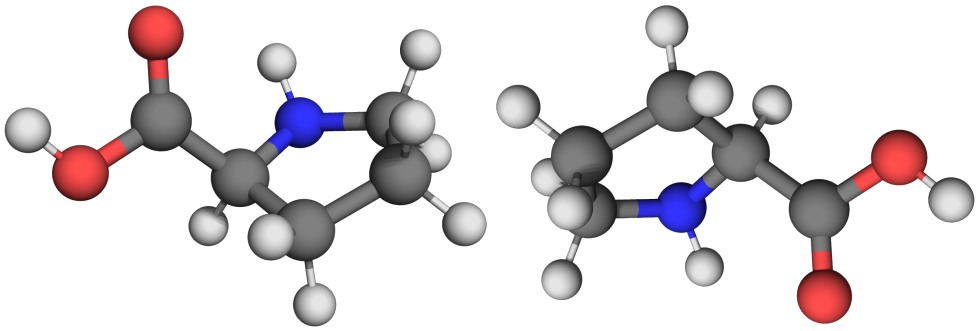


Figure 1.2. Two equivalent molecular structures of proline with different atomic coordinates.

One way to retain information about both atomic identity and molecular geometry upon rotation is the **Coulomb matrix**. For atoms with nuclear charges Z_i and Cartesian positions $\mathbf{R}_i$,

$$C_{ij} = \begin{cases} \frac{1}{2} Z_i^{2.4} & \text{for} \quad i = j, \\ \frac{Z_i Z_j}{|\mathbf{R}_i - \mathbf{R}_j|} & \text{for} \quad i \neq j. \end{cases} \tag{1.1}$$

The exponent 2.4 is empirical rather than fundamental: the diagonal term approximates the energy of an isolated atom.

Because the off-diagonal elements depend only on interatomic distances, translating or rotating the molecule does not the change Coulomb matrix. Reordering the atoms permutes the rows and columns of the matrix. An **eigenvector** is a direction that is only scaled when acted upon by a matrix: $\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$, where $\mathbf{v}$ is the eigenvector and λ is the corresponding **eigenvalue**. Therefore, while the matrix not invariant to atom ordering, its **sorted eigenspectrum** is:

$$\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \ldots, \lambda_N) \qquad \text{where} \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N. \tag{1.2}$$

Invariant does not mean unique: the Coulomb matrix eigenspectrum removes dependence on molecular orientation and atom ordering, but different molecular structures can share the same eigenspectrum. It is, therefore, not necessarily a unique molecular identifier.

Consider a water molecule with the following Cartesian coordinates (in Å):

Atom	x	y	z
O	0.0000	0.0000	0.0000
H	0.9572	0.0000	0.0000
H	-0.2400	0.9273	0.0000

Using $Z_{\mathrm{O}} = 8$ and $Z_{\mathrm{H}} = 1$.

The interatomic distances are approximately

$$r_{\mathrm{OH}_1} = 0.9572 \text{ Å}, \qquad r_{\mathrm{OH}_2} = 0.9578 \text{ Å}, \qquad r_{\mathrm{H}_1\mathrm{H}_2} = 1.514 \text{ Å}.$$

The diagonal oxygen term is

$$C_{\mathrm{OO}} = \frac{1}{2}(8)^{2.4} = 73.517,$$

while each hydrogen diagonal term is

$$C_{\mathrm{HH}} = \frac{1}{2}(1)^{2.4} = 0.500.$$

The resulting Coulomb matrix is approximately

$$\mathbf{C} = \begin{pmatrix} 73.517 & 8.358 & 8.352 \\ 8.358 & 0.500 & 0.660 \\ 8.352 & 0.660 & 0.500 \end{pmatrix}.$$

The eigenvalues are the values of λ satisfying

$$\det(\mathbf{C} - \lambda \mathbf{I}) = 0.$$

Its eigenvalues, sorted from largest to smallest, are

$$\lambda = \begin{pmatrix} 75.397 \\ -0.160 \\ -0.720 \end{pmatrix}.$$

1.3. Molecules as graphs

Three-dimensional representations are essential when the geometry of a molecule matters. For many chemical problems, however, we are primarily interested in which atoms are connected to which (**molecular topology**). This motivates representing a molecule as a **molecular graph** (Fig. 1.3).

A mathematical graph consists of a collection of **vertices** connected by **edges**. Molecular structure maps naturally onto this representation: atoms become vertices and bonds become edges. A molecular graph can contain more information than connectivity alone. Properties can be attached to both atoms and bonds. Such a graph is sometimes described as an **attributed graph**.

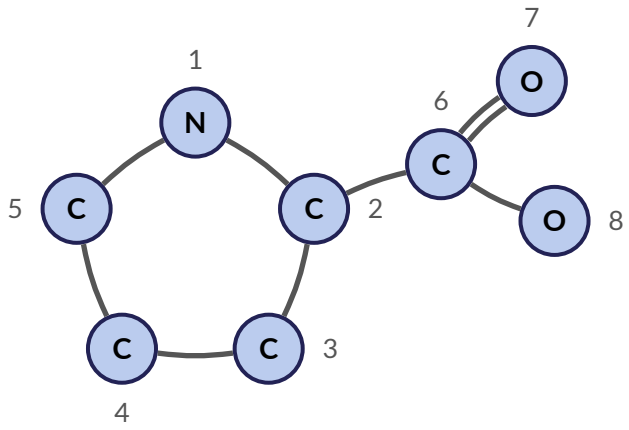


Figure 1.3. Hydrogen-suppressed molecular graph of proline. Atomic identity is represented as a vertex attribute and bond order as an edge attribute. The numerical vertex indices are arbitrary.

Once the vertices of a molecular graph have been assigned indices, the connectivity of the graph can itself be represented numerically. For a simple, unweighted molecular graph, the **adjacency matrix** $\mathbf{A}$ is defined as

We could define different numerical values for edge attributes.

$$A_{ij} = \begin{cases} 1, & \text{if vertices } i \text{ and } j \text{ are connected,} \\ 0, & \text{otherwise.} \end{cases} \tag{1.3}$$

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

1.4. Line notations

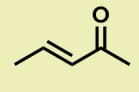
Molecular graphs provide a convenient abstract representation of chemical connectivity, but explicitly storing and displaying a graph is not always practical. An alternative is to encode the molecular graph as a sequence of characters. Such **line representations** allow molecular structures to be stored, searched and exchanged using ordinary text. The most widely encountered example is the **S**implified **M**olecular **I**nput **L**ine **E**ntry **S**ystem, or **SMILES**.

1.4.1. SMILES syntax

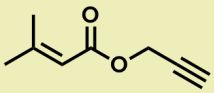
Atoms are represented using their element symbols. Adjacent atoms in the string are connected by a bond. Hydrogen atoms attached to typical organic atoms are normally omitted and inferred from standard valence rules. Single bonds are normally omitted, so for ethanol:

$$\mathrm{C{-}C{-}O} \quad \longrightarrow \quad \texttt{CCO} \tag{1.4}$$

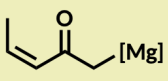
Double and triple bonds are written explicitly using an equals sign (`O=C=O` is carbon dioxide) and a hash symbol (`C#C` is acetylene). A branch in the molecular graph is written inside parentheses following the atom (`CC(=O)N` is acetamide). Tetrahedral stereochemistry can be included using `@` and `@@`, interpreted relative to the order of neighbouring atoms in the string. Double-bond stereochemistry can be represented using directional bond symbols.



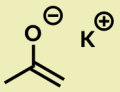
`C/C=C/C(=O)C`



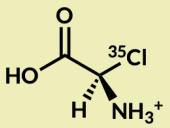
`C/C(C)=C/C(=O)OCC#C`



`C/C=C\C(=O)C[Mg]`



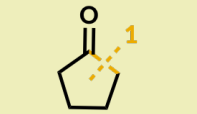
`CC([O-])=C.[K+]`



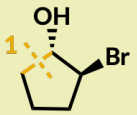
`OC([C@H]([NH3+])[35Cl])=O`

A cyclic molecular graph cannot be represented simply by walking along a sequence of atoms without eventually reconnecting two vertices. SMILES indicates this **ring closure** using matching numerical labels. Aromatic atoms can be represented either as Kekulé structure or with lower-case letters.

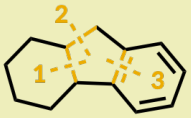
The ring number does not represent an atom. Matching ring labels indicate the edge connecting two atoms encountered during the graph traversal.



`O=C1CCCC1`

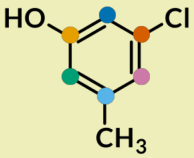


`O[C@@H]1[C@@H](Br)CCC1`



`C12CCCCC1c3ccccc3C2`

A molecular graph can usually be traversed in more than one way. Consequently, a single molecule can have multiple valid SMILES representations.



`OC1=CC(Cl)=CC(C)=C1`

`C1(Cl)=CC(O)=CC(C)=C1`

`Cc1cc(O)cc(Cl)c1`

For database searching and comparison, having many valid strings for the same molecular graph is inconvenient. A **canonicalisation algorithm** assigns a reproducible ordering to the graph and generates a corresponding canonical SMILES.

Canonical does not mean universal. Canonical SMILES are generated algorithmically. Different software packages can employ different canonicalisation procedures, so the same molecule need not have one universally defined canonical SMILES. These can even vary between software versions!

Several related line notations extend or modify the ideas used in SMILES. **SMARTS** describes a molecular pattern rather than one specific molecule. It is used for substructure searching, allowing questions such as whether a molecule contains an alcohol, amide or aromatic nitrogen atom. **SMIRKS** encodes a chemical transformation by specifying patterns in the reactants and the corresponding patterns in the products. **SELFIES** is a string representation designed so that every valid SELFIES string corresponds to a molecular graph permitted by its underlying rules. This makes SELFIES useful in generative machine learning, where arbitrary modification of a SMILES will often produce an invalid structure.

1.5. Identifiers and chemical records

SMILES strings can also contain special characters that are inconvenient when structures are passed between different pieces of software or used as search terms. More fundamentally, the existence of multiple valid SMILES representations makes them inconvenient as universal chemical identifiers for cross-referencing structures from different sources. For searching databases and linking chemical information between different sources, it is useful to have a standardised molecular identifier.

1.5.1. The International Chemical Identifier (InChI)

To address these problems, IUPAC developed the **International Chemical Identifier (InChI)**. InChI is a structure-based, non-proprietary, open-source, and standardised identifier in which different types of chemical information are organised hierarchically into layers. This makes InChI particularly useful when chemical information from different sources needs to be linked or incorporated into **FAIR** (**f**indable, **a**ccessible, **i**nteroperable, and **r**eusable) datasets.

An InChI is organised into layers of chemical information. An InChI begins with a version identifier and molecular formula, followed by additional layers describing features such as connectivity, hydrogen atoms, charge, stereochemistry, and isotopic composition. Layers are included only where the corresponding information is required to describe the structure.

The ability to read and parse InChI by humans was of little importance to the developers. You are not expected to construct an InChI manually. You should, however, be able to recognise the organisation of an InChI and identify where important types of chemical information are stored (Fig. 1.4).

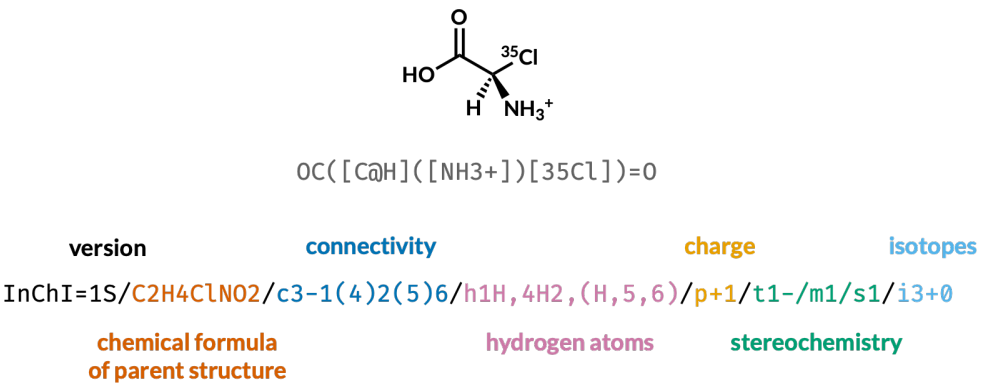


Figure 1.4. An example chemical structure with its corresponding InChI. Note how each layer gives more and more chemical information.

1.5.2. Search engine and indexing: the InChIKey

Although InChIs provide standardised identifiers, they can have varying lengths and contain characters that are inconvenient for search engines, database indexing and filenames. For this reason, a second identifier, the **InChIKey**, can be generated from the InChI. An InChIKey is a hash of an InChI that produces a compact 27-character identifier containing uppercase letters and hyphens (Fig. 1.5).

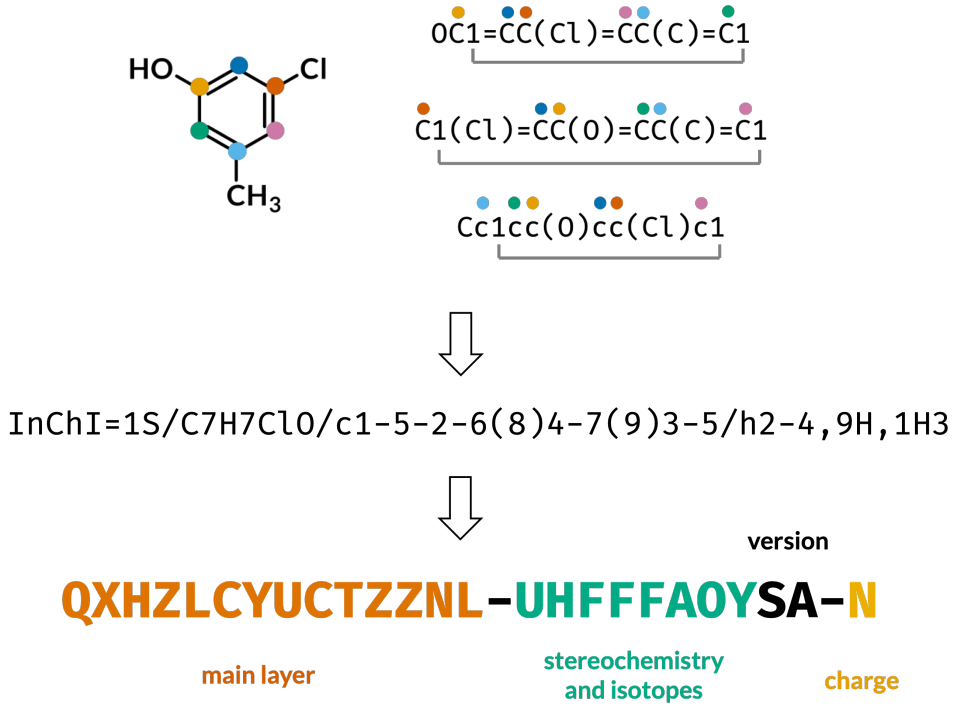
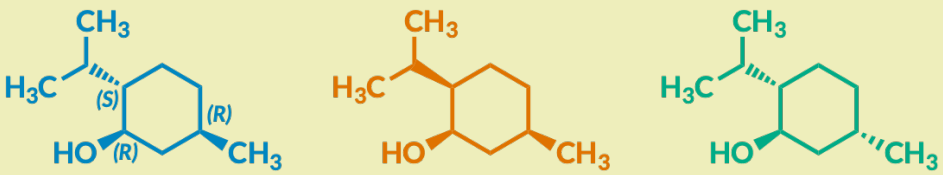


Figure 1.5. Different molecular representations of the same molecule: non-unique SMILES, standardised InChI and its corresponding InChIKey.

> Hashing is a one-way operation. The same InChI will reproducibly generate the same InChIKey, but it is not possible to recover the complete molecular structure from the InChIKey alone. While they are convenient filenames, it is wise to keep a 'dictionary' mapping them to structural information.

The InChIKey is divided into blocks. The first block represents information associated primarily with connectivity, while subsequent parts encode information associated with stereochemistry, isotopic composition and protonation.

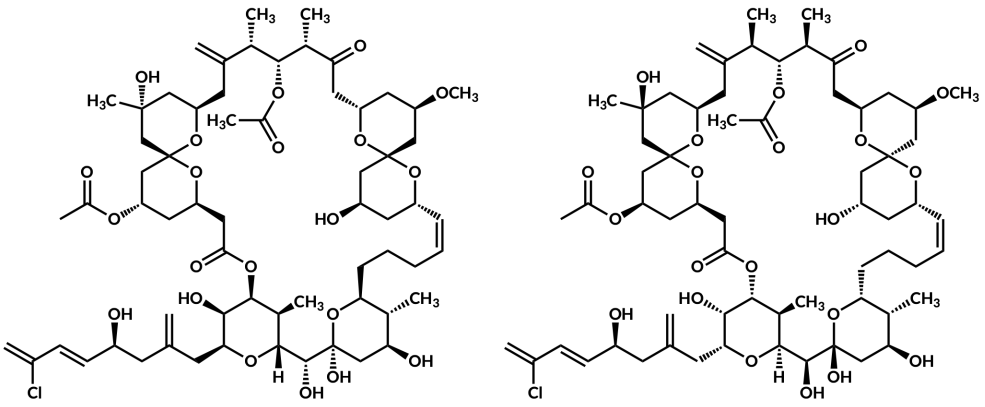


NOOLISFMXDJSKH-KXUCPTDWSA-N

NOOLISFMXDJSKH-OPRDCNLKSA-N

NOOLISFMXDJSKH-LPEHRKFASA-N

Compressing an InChI into a much shorter InChIKey necessarily loses information. This creates the possibility of a *hash collision*, in which different InChIs generate the same InChIKey. While the `SHA-256 hash` was chosen to minimise collisions of InChIKeys, they are still possible – especially for large structures. Consider two diastereomers of Spongistatin I, a molecule with 26 stereocentres, which have different InChI but the same InChIKey (Fig. 1.6). Such collisions are incredibly rare, but it is prudent to check they have not happened in our datasets!



ICXJVZHDZFXYQC-RAZYNMGUSA-N

2^{26} = 67,108,864 diastereomers!

Figure 1.6. Two diastereomers of Spongistatin I with the same InChIKey.

1.6. Bit vectors and molecular fingerprints

So far, we have considered representations that aim to describe or identify a molecular structure. For many cheminformatics applications, however, we do not need to reconstruct the complete structure of a molecule. Instead, we may want to answer much simpler questions: does a particular structural feature occur in the molecule, and how similar is one molecule to another? For these tasks, it is useful to convert molecular structures into compact numerical representations known as **molecular fingerprints**.

A **bit vector** is a one-dimensional array containing zeros and ones, for example,

$$\mathbf{x} = (0, 0, 1, 1, 0, 1). \tag{1.5}$$

Each position, or *bit*, can be used to record the presence or absence of some feature of a higher-dimensional object such as a molecule. This might initially seem an unnecessarily crude way of representing chemical structure. However, identifying and comparing substructures directly across very large molecular databases can be computationally expensive. Once molecules have been converted into bit vectors, computers can compare their patterns of zeros and ones extremely efficiently.

> **Molecular fingerprints trade detail for computational efficiency.** A molecular fingerprint encodes selected features of a molecular structure in a compact numerical form. It is generally not possible to reconstruct the complete molecular structure from the fingerprint.

1.6.1. Structural keys

One simple way to construct such a bit vector is to define a fixed list of yes/no structural questions. Each question is assigned a position in the bit vector, and the corresponding bit is switched on if the feature is present. A very simple list of such structural questions could be:

- does the molecule contain oxygen?
- does it contain a ring?
- does it contain an aromatic atom?
- does it contain a six-membered ring?
- does it contain a carbonyl group?

A structural-key fingerprint asks a predefined set of questions about a molecule. Each structural feature is associated with a particular bit:

$$\text{feature absent} \longrightarrow 0$$
$$\text{feature present} \longrightarrow 1$$

Two molecules sharing many of the encoded structural features will consequently share many 'on' bits, which can be cheaply compared.

One widely used example is the **M**olecular **ACC**ess **S**ystem (**MACCS**) structural key fingerprint. MACCS keys use a predefined collection of 166 structural patterns intended to summarise useful information about molecular structure (Fig. 1.7).

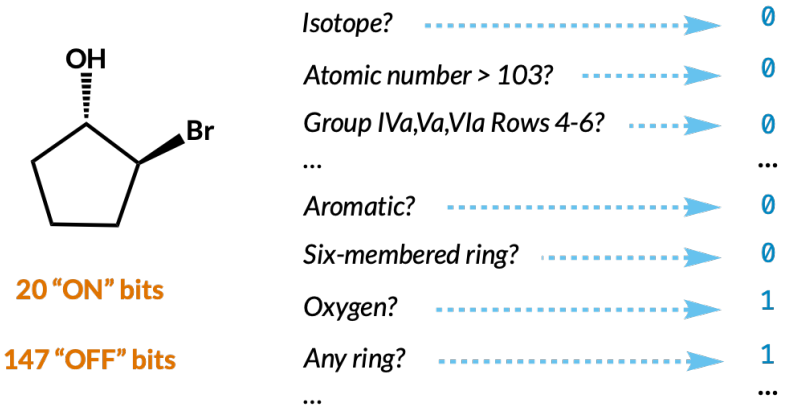


Figure 1.7. Example MACCS keys for a simple organic molecule.

Someone must decide in advance which structural patterns are worth encoding. A set of keys designed for one region of chemistry may fail to capture the structural distinctions that matter in another.

1.6.2. Circular fingerprints

Rather than deciding in advance which structural patterns are important, we can generate features directly from the molecular graph. **Circular fingerprints** describe the local environment surrounding each atom and progressively expand that environment through neighbouring atoms:

1. assign an initial identifier to each atom;
2. identify the neighbouring atoms and bonds;
3. combine this information to describe progressively larger local environments;
4. convert each environment into a numerical identifier;
5. encode the resulting identifiers in a fingerprint.

The size of an atomic environment is described by its **radius**. At radius 0, the environment describes the atom itself. Radius 1 additionally includes directly bon-

ded neighbours, radius 2 extends to the next shell of atoms, and so forth (Fig. 1.8). Increasing the radius incorporates progressively more of the local molecular environment surrounding each atom.

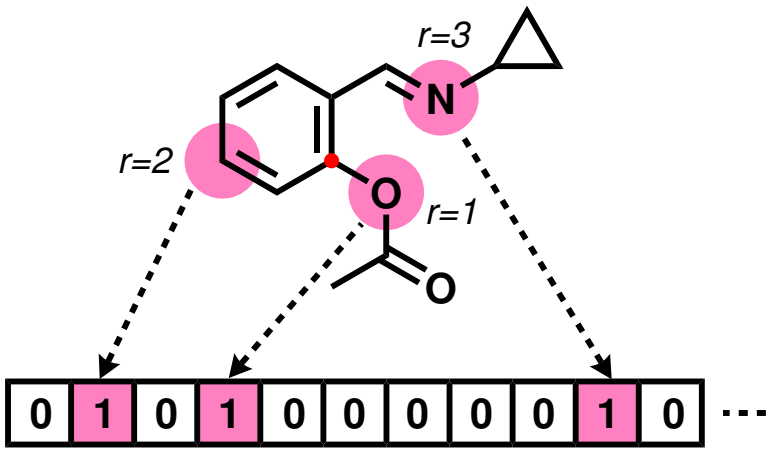


Figure 1.8. Conceptual representation of a circular fingerprint.

One of the most widely used approaches to circular fingerprints derives from the Morgan algorithm introduced by the Chemical Abstracts Society (CAS). A more recent development of this idea is the **Extended-Connectivity Fingerprint (ECFP)**. For each atom, the algorithm generates identifiers describing its local molecular environments. These identifiers can then be **hashed**: a numerical operation converts an identifier into an integer, which can subsequently be mapped onto a position in a fingerprint of fixed length through modulo division (Fig. 1.9). If the same structural environment occurs in another molecule, the same procedure generates the same identifier and therefore maps to the same position in the fingerprint. Different substructures can give rise to the same bits, so while a fingerprint can easily confirm the absence of a substructure, it does not indicate any certainty that the substructure is indeed present. These **hash collisions** are what you see in spurious substructures in CAS SciFinder.

> **Morgan fingerprints describe which local molecular environments occur within a molecular graph.** Unlike predefined structural keys, those environments do not need to be chosen before the molecules are analysed, making circular fingerprints useful across a wide range of chemical datasets.

A binary fingerprint records only whether a feature occurs:

$$x_i \in 0, 1. \tag{1.6}$$

An alternative is a **count fingerprint**, in which each position records how many times a feature occurs within the molecule:

$$x_i \in 0, 1, 2, ... \tag{1.7}$$

Count fingerprints therefore retain information that is lost when a bit is simply

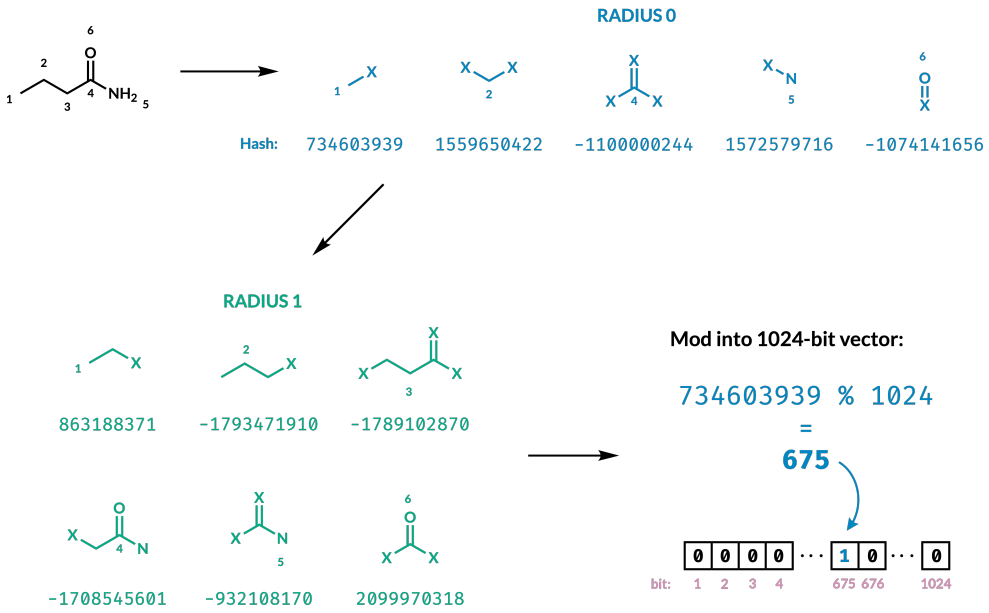


Figure 1.9. Generation of an ECFP fingerprint with contributions from environments at different radii. Bits activated by each environment are based on the hashing function.

switched on. Extended-connectivity fingerprints normally distinguish atomic environments using atom-specific information. A related representation, the **Functional-Class Fingerprint (FCFP)**, instead groups atoms according to chemically relevant roles, such as hydrogen-bond donor or acceptor, aromatic character, or membership of broader functional classes.

1.7. Summary

In this chapter, we have seen that a molecule can be represented digitally in many different ways, and that **there is no universally 'best' molecular representation**. Instead, the appropriate representation depends on the chemical information that must be retained and the task we want to perform. Cartesian coordinates provide detailed information about **molecular geometry**, while molecular graphs abstract a structure into its **atoms and connectivity**. Line representations such as SMILES provide a compact way of encoding molecular graphs as text, while InChI and InChIKey provide standardised approaches to **chemical identification and data interoperability**. Finally, molecular fingerprints deliberately discard some structural information to produce compact numerical representations that are efficient to store and compare. These representations therefore illustrate a recurring theme in digital chemistry: **the choice of representation determines which chemical information is made explicit, which information is hidden, and which information is lost**. Having learnt how to represent individual molecules, we can now ask a different question: if molecules are represented computationally, **how can we decide how similar or different they are?** Answering this question takes us from individual molecular representations to the organisation and exploration of the **chemical space**.

2. Exploring chemical space

2.1. Introduction

This chapter develops the role of exploring chemical space within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

2.2. What is chemical space?

Keywords: chemical space; scope; task.

2.2.1. Theoretical and accessible spaces

2.2.2. Task-specific spaces: substrates, ligands, catalysts and solvents

2.3. Molecular similarity and distance

Keywords: distance; Tanimoto; neighbours.

2.3.1. Similarity is representation-dependent

2.3.2. Tanimoto similarity

2.3.3. Nearest-neighbour searching

2.4. Structural and property-based spaces

Keywords: structure; properties; geometry.

2.4.1. Fingerprint space

2.4.2. Physicochemical-property space

2.4.3. Electronic and steric ligand space

2.5. High-dimensional data

Keywords: dimensions; scaling; correlation.

2.5.1. The curse of dimensionality

2.5.2. Scaling and correlated variables

2.6. Principal component analysis

Keywords: PCA; scores; loadings.

2.6.1. Variance and covariance

2.6.2. Principal components and projection

2.6.3. Scores and loadings

2.6.4. Explained variance and interpretation

2.7. Other dimensionality-reduction methods

Keywords: local structure; visualisation; distortion.

2.7.1. t-SNE

2.7.2. UMAP

2.7.3. When an attractive map misleads

2.8. Clustering chemical data

Keywords: clusters; centroids; diversity.

2.8.1. k-means clustering

2.8.2. Hierarchical clustering

2.8.3. Selecting representative compounds

2.9. Green chemistry case study: solvent space

Keywords: solvent space; PCA; sustainability.

2.9.1. Why solvent substitution is a chemical-space problem

2.9.2. The ACS solvent selection map

2.9.3. Finding functionally similar alternatives

2.9.4. Adding health, safety and environmental criteria

2.9.5. Similarity does not imply sustainability

2.10. Designing informative datasets

Keywords: coverage; bias; data quality.

2.10.1. Coverage, diversity and bias

2.10.2. Train/test coverage

2.10.3. Missing values, duplicates and failed experiments

2.11. Chapter summary and practice

Keywords: similarity; diversity; decision.

2.11.1. Select a diverse screening set

2.11.2. Propose and critique a solvent substitution

2.12. Summary

Key take-away: complete after drafting the chapter.

2.13. Questions for review

3. Visual methods for mechanistic investigations

3.1. Introduction

This chapter develops the role of visual methods for mechanistic investigations within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

3.2. Seeing mechanism in experimental data

Keywords: visual analysis; hypothesis; deviation.

3.2.1. Plots as tests of chemical hypotheses

3.2.2. Expected relationships and diagnostic deviations

3.3. Linear free-energy relationships

Keywords: LFER; Hammett; mechanism.

3.3.1. Hammett relationships

3.3.2. Meaning of slope and intercept

3.3.3. Interpreting deviations from linearity

3.4. Asymmetric catalysis and non-linear effects

Keywords: ee; NLE; aggregation.

3.4.1. Catalyst ee and product ee

3.4.2. Positive and negative non-linear effects

3.4.3. Aggregation models

3.4.4. Alternative origins of non-linearity

3.5. Proline organocatalysis as chemical context

Keywords: proline; enamine; stereoselectivity.

3.5.1. Enamine catalysis

3.5.2. Stereochemical induction

3.5.3. What kinetic and selectivity data can reveal

3.6. Kagan, Soai and asymmetric amplification

Keywords: Kagan; Soai; autocatalysis.

3.6.1. Non-linear effects in asymmetric catalysis

3.6.2. Asymmetric autocatalysis

3.6.3. Amplification and symmetry breaking

3.7. Reaction progress kinetics

Keywords: initial rates; profiles; experiment design.

3.7.1. Limitations of initial rates

3.7.2. Concentration-time and rate profiles

3.7.3. Designing informative kinetic experiments

3.8. Reaction progress kinetic analysis

Keywords: RPKA; same excess; different excess.

3.8.1. Same-excess experiments

3.8.2. Catalyst deactivation and product inhibition

3.8.3. Different-excess experiments

3.8.4. Determining reaction orders

3.9. Variable time normalisation analysis

Keywords: VTNA; normalised time; overlay.

3.9.1. Normalised time

3.9.2. Finding reactant order

3.9.3. Finding catalyst order

3.9.4. Interpreting successful and failed overlays

3.10. Mechanistic case study

Keywords: orders; inhibition; proposal.

3.10.1. Kinugasa reaction dataset

3.10.2. From profile overlays to a mechanistic proposal

3.11. What has the evidence established?

Keywords: evidence; falsification; ambiguity.

3.11.1. Supporting and falsifying mechanisms

3.11.2. Non-uniqueness and complementary experiments

3.12. Chapter summary and practice

Keywords: plot; diagnose; infer.

3.12.1. Interpret an NLE plot

3.12.2. Diagnose RPKA profiles

3.12.3. Select a VTNA order

3.13. Summary

Key take-away: complete after drafting the chapter.

3.14. Questions for review

4. Machine learning for chemistry: a primer

4.1. Introduction

This chapter develops the role of machine learning for chemistry: a primer within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

4.2. From physical organic models to machine learning

Keywords: LFER; features; prediction.

4.2.1. From Hammett to multivariate relationships

4.2.2. Features, targets and models

4.3. Types of learning problem

Keywords: supervised; unsupervised; target.

4.3.1. Regression

4.3.2. Classification

4.3.3. Clustering and dimensionality reduction

4.3.4. Prediction versus causal explanation

4.4. Chemical datasets

Keywords: rows; columns; preprocessing.

4.4.1. Observations and features

4.4.2. Missing and categorical data

4.4.3. Scaling and preprocessing

4.5. Training, validation and testing

Keywords: validation; leakage; split.

4.5.1. Train/validation/test partitions

4.5.2. Cross-validation

4.5.3. Chemically meaningful data splits

4.6. Linear regression

Keywords: coefficients; residuals; regularisation.

4.6.1. Univariate and multivariate regression

4.6.2. Coefficients and residuals

4.6.3. Regularisation as a concept

4.7. Learning non-linear relationships

Keywords: non-linearity; ensemble; flexibility.

4.7.1. Decision trees

4.7.2. Random forests

4.7.3. Kernel and neural-network concepts

4.8. Evaluating models

Keywords: error; baseline; residuals.

4.8.1. Baselines

4.8.2. MAE, RMSE and R-squared

4.8.3. Classification metrics

4.8.4. Residual plots

4.9. Overfitting and underfitting

Keywords: overfitting; leakage; y-scrambling.

4.9.1. Bias and variance

4.9.2. Model complexity

4.9.3. Data leakage and chance correlation

4.10. Applicability domain and uncertainty

Keywords: domain; extrapolation; uncertainty.

4.10.1. Interpolation and extrapolation

4.10.2. Connection to chemical-space coverage

4.10.3. Prediction confidence and model uncertainty

4.11. Green chemistry case study: solvent effects

Keywords: solvent effects; kinetics; prioritisation.

4.11.1. Predicting solvent-dependent reaction behaviour

4.11.2. Useful target versus convenient target

4.11.3. Using predictions to prioritise experiments

4.12. Chapter summary and practice

Keywords: task; validate; critique.

4.12.1. Choose a learning task

4.12.2. Critique a validation strategy

4.12.3. Read a residual plot

4.13. Summary

> **Key take-away:** complete after drafting the chapter.

4.14. Questions for review

5. Modern molecular descriptors

5.1. Introduction

This chapter develops the role of modern molecular descriptors within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

5.2. From representation to descriptor

Keywords: descriptor; feature; information.

5.2.1. Representation, descriptor and feature

5.2.2. What should the model know?

5.3. Classical physical-organic descriptors

Keywords: electronic effects; steric effects; LFER.

5.3.1. Hammett substituent constants

5.3.2. Taft parameters

5.3.3. Scope and transferability

5.4. Steric descriptors

Keywords: Sterimol; buried volume; shape.

5.4.1. Cone angle and A-values

5.4.2. Sterimol parameters

5.4.3. Buried volume

5.4.4. Solvent-accessible surface area

5.5. Conformational dependence

Keywords: conformers; weighting; flexibility.

5.5.1. Single structures and ensembles

5.5.2. Minimum, maximum and Boltzmann-weighted values

5.5.3. Flexibility as information

5.6. Electronic descriptors

Keywords: HOMO; charges; polarity.

5.6.1. Frontier-orbital energies

5.6.2. Charges and electrostatic potentials

5.6.3. Dipole and polarisability

5.6.4. Vibrational and spectroscopic quantities

5.7. Descriptor generation at scale

Keywords: cost; level of theory; standardisation.

5.7.1. Experimental, DFT and semi-empirical routes

5.7.2. Computational cost and reproducibility

5.7.3. Descriptor libraries and Kraken-type spaces

5.8. Fingerprints versus physicochemical descriptors

Keywords: fingerprints; interpretability; sparsity.

5.8.1. Interpretability

5.8.2. Dimensionality and sparsity

5.8.3. 2D and 3D information

5.9. Learned representations

Keywords: GNN; message passing; representation learning.

5.9.1. Attributed molecular graphs

5.9.2. Message passing

5.9.3. Hand-crafted versus learned features

5.10. Choosing descriptors for a task

Keywords: task fit; selection; leakage.

5.10.1. Similarity search

5.10.2. Mechanistic modelling

5.10.3. Large-data prediction

5.10.4. Avoiding redundant and leaked features

5.11. Chapter summary and practice

Keywords: compare; select; justify.

5.11.1. Compare descriptor families

5.11.2. Design a feature set

5.12. Summary

> **Key take-away:** complete after drafting the chapter.

5.13. Questions for review

6. Reaction optimisation: OVAT, DoE and Bayesian optimisation

6.1. Introduction

This chapter develops the role of reaction optimisation: ovat, doe and bayesian optimisation within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

6.2. Defining an optimisation problem

Keywords: factors; constraints; objective.

6.2.1. Variables, constraints and objectives

6.2.2. Continuous and categorical factors

6.2.3. What counts as an optimum?

6.3. One variable at a time

Keywords: OVAT; interactions; local optimum.

6.3.1. The OVAT workflow

6.3.2. Strengths and limitations

6.3.3. Hidden interactions

6.4. Factorial experiments

Keywords: factorial; effects; interactions.

6.4.1. Factors and levels

6.4.2. Main effects and interactions

6.4.3. Full and fractional factorial designs

6.5. Response surfaces and design of experiments

Keywords: DoE; response surface; curvature.

6.5.1. Empirical response surfaces

6.5.2. Curvature and non-linear response

6.5.3. Efficient information gathering

6.6. Sequential experimentation

Keywords: adaptive design; iteration; information.

6.6.1. Learning from each round

6.6.2. Batch and sequential choices

6.7. Bayesian optimisation

Keywords: surrogate; acquisition; uncertainty.

6.7.1. Surrogate models

6.7.2. Prediction and uncertainty

6.7.3. Acquisition functions

6.7.4. Exploration and exploitation

6.8. Green multi-objective optimisation

Keywords: PMI; sustainability; Pareto front.

6.8.1. Yield, selectivity and productivity

6.8.2. PMI, waste, energy, hazard and cost

6.8.3. Pareto trade-offs

6.9. Failed and infeasible experiments

Keywords: failure; feasibility; constraints.

6.9.1. Failures as data

6.9.2. Constraints and safe operating regions

6.10. Worked reaction-optimisation case study

Keywords: conditions; evidence; next experiment.

6.10.1. Define the search space

6.10.2. Compare OVAT, DoE and BO recommendations

6.10.3. Select the next experiment

6.11. Chapter summary and practice

Keywords: choose; trade off; justify.

6.11.1. Choose an optimisation strategy

6.11.2. Interpret interactions

6.11.3. Balance competing objectives

6.12. Summary

Key take-away: complete after drafting the chapter.

6.13. Questions for review

7. Generating chemical data: high-throughput and flow chemistry

7.1. Introduction

This chapter develops the role of generating chemical data: high-throughput and flow chemistry within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

7.2. Why generate data at scale?

Keywords: throughput; information; purpose.

7.2.1. Screening, understanding and optimisation

7.2.2. Throughput versus information

7.3. Parallel experimentation

Keywords: parallel; liquid handling; scale.

7.3.1. Reaction arrays and plates

7.3.2. Liquid handling and dosing

7.3.3. Miniaturisation

7.4. High-throughput reaction screening

Keywords: screen; library; coverage.

7.4.1. Substrates, ligands, catalysts and conditions

7.4.2. Experimental design for screening

7.5. The analytical bottleneck

Keywords: analysis; calibration; bottleneck.

7.5.1. Sampling strategies

7.5.2. LC, GC, MS and spectroscopy

7.5.3. Calibration and automated data processing

7.6. Flow chemistry fundamentals

Keywords: flow; residence time; steady state.

7.6.1. Reactor architecture

7.6.2. Residence time

7.6.3. Mixing and heat transfer

7.6.4. Steady state

7.7. Serial experimentation in flow

Keywords: serial; lag; inline analysis.

7.7.1. Changing conditions

7.7.2. Accounting for system lag and dispersion

7.7.3. Integrated analysis

7.8. Batch versus flow automation

Keywords: batch; flow; task fit.

7.8.1. Matching architecture to the chemical problem

7.8.2. Advantages, limitations and hybrid systems

7.9. Data infrastructure

Keywords: provenance; standards; QC.

7.9.1. Metadata and provenance

7.9.2. Machine-readable experimental records

7.9.3. Failed experiments and quality control

7.10. Does automation make chemistry greener?

Keywords: resource use; waste; evidence.

7.10.1. Miniaturisation and process intensification

7.10.2. Solvent, analytical and consumables burdens

7.10.3. Measuring rather than assuming greenness

7.11. Chapter summary and practice

Keywords: generate; measure; record.

7.11.1. Design a data-generation workflow

7.11.2. Identify the bottleneck

7.11.3. Compare batch and flow

7.12. Summary

Key take-away: complete after drafting the chapter.

7.13. Questions for review

8. Autonomous and self-driving laboratories

8.1. Introduction

This chapter develops the role of autonomous and self-driving laboratories within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

8.2. Automation, autonomy and closed loops

Keywords: automation; autonomy; closed loop.

8.2.1. Degrees of automation

8.2.2. High throughput versus autonomy

8.2.3. Human and machine decisions

8.3. The experimental closed loop

Keywords: represent; decide; update.

8.3.1. Represent

8.3.2. Choose

8.3.3. Execute

8.3.4. Measure

8.3.5. Learn and update

8.4. Hardware layer

Keywords: hardware; sensors; instruments.

8.4.1. Robots, pumps and reactors

8.4.2. Sensors and analytical instruments

8.5. Software and data layer

Keywords: control; orchestration; interoperability.

8.5.1. Instrument control

8.5.2. Orchestration

8.5.3. Data standards and interoperability

8.6. Decision layer

Keywords: BO; active learning; objectives.

8.6.1. DoE and Bayesian optimisation

8.6.2. Machine learning and active learning

8.6.3. Multi-objective decisions

8.7. Closing the loop

Keywords: feedback; update; iteration.

8.7.1. Triggering experiments

8.7.2. Processing results

8.7.3. Updating the model and selecting the next experiment

8.8. Robustness and error handling

Keywords: faults; anomalies; recovery.

8.8.1. Hardware faults

8.8.2. Analytical anomalies

8.8.3. Chemical failures and recovery

8.9. Human-in-the-loop chemistry

Keywords: judgement; oversight; trust.

8.9.1. Defining objectives and constraints

8.9.2. Chemical judgement and intervention

8.9.3. Trust and explainability

8.10. Green autonomous chemistry

Keywords: objective; sustainability; responsibility.

8.10.1. Optimising more than yield

8.10.2. Embedding PMI, energy, hazard and cost

8.10.3. The objective function encodes priorities

8.11. What should laboratories automate?

Keywords: bottleneck; value; limits.

8.11.1. Scientific bottlenecks

8.11.2. Reproducibility and scale

8.11.3. Hype, limitations and responsible deployment

8.12. Course synthesis and outlook

Keywords: connect; integrate; critique.

8.12.1. Connecting all eight lectures

8.12.2. FAIR data, connected instruments and AI/ML

8.12.3. Digital chemistry as scalable chemical reasoning

8.13. Summary

Key take-away: complete after drafting the chapter.

8.14. Questions for review

9. Workshop: molecules as data with RDKit

9.1. Introduction

This chapter develops the role of workshop: molecules as data with rdkit within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

9.2. Workshop aims and setup

Keywords: RDKit; notebook; molecule object.

9.2.1. Notebook structure

9.2.2. SMILES input and molecule objects

9.3. Drawing and inspecting molecules

Keywords: draw; inspect; query.

9.3.1. Two-dimensional depiction

9.3.2. Atoms, bonds and molecular properties

9.4. Substructure searching

Keywords: SMARTS; match; substructure.

9.4.1. SMARTS patterns

9.4.2. Matching functional groups

9.5. Fingerprints and similarity

Keywords: Morgan; Tanimoto; ranking.

9.5.1. Morgan fingerprints

9.5.2. Tanimoto similarity

9.5.3. Ranking neighbours

9.6. Calculating simple descriptors

Keywords: descriptors; table; properties.

9.6.1. Molecular weight, polarity and hydrogen bonding

9.6.2. Building a small descriptor table

9.7. Selecting a diverse mini-library

Keywords: diversity; selection; justification.

9.7.1. Pairwise similarity

9.7.2. Representative selection

9.7.3. Chemical justification

9.8. Workshop reflection

Keywords: assumptions; interpretation; limits.

9.8.1. Representation choices

9.8.2. What RDKit did and did not establish

9.9. Summary

> **Key take-away:** complete after drafting the chapter.

9.10. Questions for review

10. Revision class and example examination questions

10.1. Introduction

This chapter develops the role of revision class and example examination questions within the course narrative.

Chapter thread: motivation; method; chemical application; limitations; connection.

10.2. How the course fits together

Keywords: connect; integrate; transfer.

10.2.1. Represent to explore

10.2.2. Understand to model

10.2.3. Decide to execute

10.2.4. Closing the loop

10.3. How to approach examination questions

Keywords: identify; justify; interpret.

10.3.1. Identify the chemical decision

10.3.2. Select and justify a method

10.3.3. State assumptions and limitations

10.3.4. Interpret rather than describe

10.4. Example question: representation and chemical space

Keywords: representation; PCA; solvent choice.

10.4.1. Choose a representation

10.4.2. Interpret a PCA map

10.4.3. Propose a greener solvent substitute

10.5. Example question: mechanism from data

Keywords: NLE; RPKA; evidence.

10.5.1. Interpret NLE, RPKA and VTNA evidence

10.5.2. State what is and is not established

10.6. Example question: modelling and validation

Keywords: features; validation; domain.

10.6.1. Choose descriptors and a model

10.6.2. Critique a data split

10.6.3. Assess extrapolation

10.7. Example question: optimisation and automation

Keywords: optimise; automate; sustain.

10.7.1. Compare OVAT, DoE and BO

10.7.2. Design a closed-loop workflow

10.7.3. Include sustainability objectives

10.8. Final checklist

Keywords: recall; apply; critique.

10.8.1. Definitions

10.8.2. Plots and diagnostics

10.8.3. Method selection

10.8.4. Limitations and connections

10.9. Summary

Key take-away: complete after drafting the chapter.

10.10. Questions for review